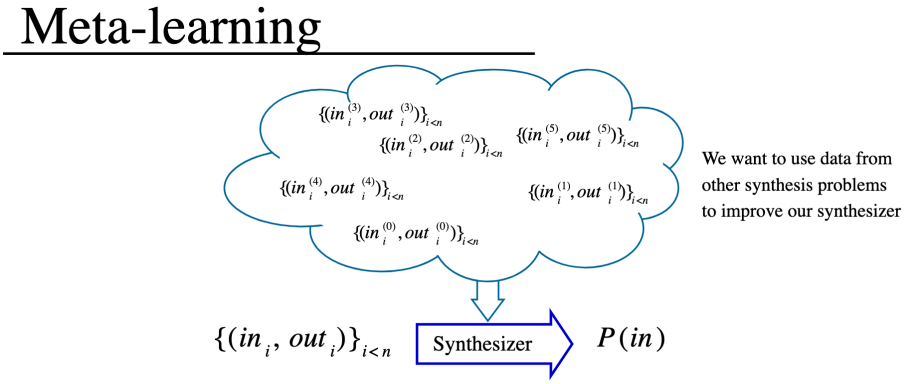
LECTURE 9

Component Discovery

As we have seen, search-based program synthesis can be a very powerful tool, especially when combined with various ways of pruning and efficiently exploring the search space. However, one thing we have not yet paid much attention to is the role of the domain-specific language (DSL) itself. The choice of DSL is arguably the most important choice in the use of search-based synthesis. First, the DSL determines the speed and scalability of the synthesis process, both by directly determining the size of the search space, and indirectly by influencing the effectiveness of the various search optimizations we have discussed. Additionally, the DSL determines how the synthesized programs will generalize to unseen inputs, because it determines what programs can be represented concisely. The goal in DSL design is to find a set of primitives that is expressive enough to allow us to express the desired programs concisely, but one where undesired programs are not expressible at all, or only in a very verbose manner.

In this lecture, we are going to explore the idea of *component discovery*, which will allow us to start from a low-level DSL and automatically discover new components that can be added to the DSL both to improve the efficiency of the synthesis process and to help the synthesized programs generalize better.



Synthesis as a learning problem

Component discovery can be understood as a form of *meta-learning*. The general idea of meta-learning is that a system learns to learn more efficiently. If we view each inductive program synthesis problem as a learning problem — we are trying to learn a program that fits the provided examples — then

component discovery allows us to learn from a collection of sample synthesis problems a set of components that can help us more efficiently solve future synthesis problems.

9.1 Some history

Back in the 1980s, the cutting edge in artificial intelligence were expert systems, which codified knowledge in the form of logical predicates and could then use deduction to derive new facts from known ones. Practitioners very quickly realized, however, that there actually wasn't much intelligence in this approach to AI; much like a traditional computer program, the expert systems were a good way to capture and deploy existing knowledge, but they were not very good at discovering new knowledge. In the late 1980s, a group of researchers, with Stephen Muggleton playing a leading role, started focusing on the problem of how to automatically learn new logical predicates from examples. These explorations eventually gave rise to the field of *inductive logic programming* (ILP) [1].

One of the core problems in ILP is the *predicate invention* problem, which involves automatically discovering new predicates that can be used to represent the concepts in the training data more effectively. In a seminal 1987 paper, Muggleton [2] showed that new predicates could be created by identifying common patterns in existing predicates. For example, given two concepts:

$$X \leftarrow B \wedge C \wedge D \wedge E$$
$$Y \leftarrow A \wedge B \wedge D \wedge F$$

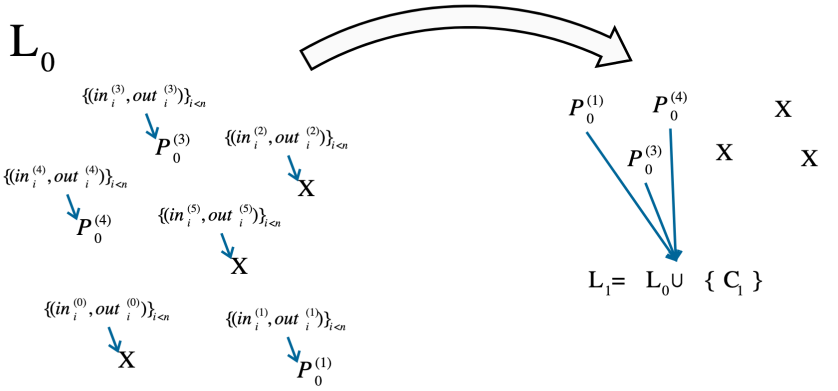
You could introduce a new concept $Z \leftarrow B \wedge D$ and rewrite the original concepts as:

$$X \leftarrow Z \wedge C \wedge E$$
$$Y \leftarrow A \wedge Z \wedge F$$

and this new concept Z would then be available for use in future concepts. This was one of six rules that the paper introduced for predicate invention, and then it defined a search algorithm that would explore how to apply these rules with the goal of producing a minimal representation of the original set of rules. For example, given a set of rules for identifying different animal types in terms of their basic features, (e.g. has feathers, has beak, lays eggs, etc.), the algorithm could discover higher-level concepts such as "bird" or "mammal" that would then be useful in more compactly representing many different kinds of specific birds or mammals.

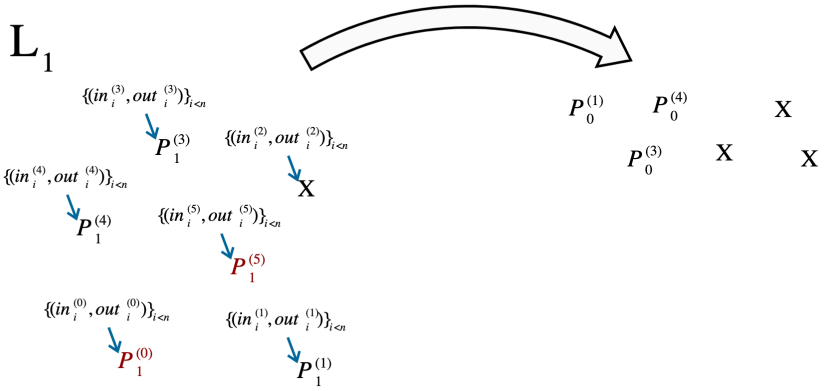
Exploration Compression Algorithm

Dechter, Malmaud, Adams & Tenenbaum (IJCAI 2013)



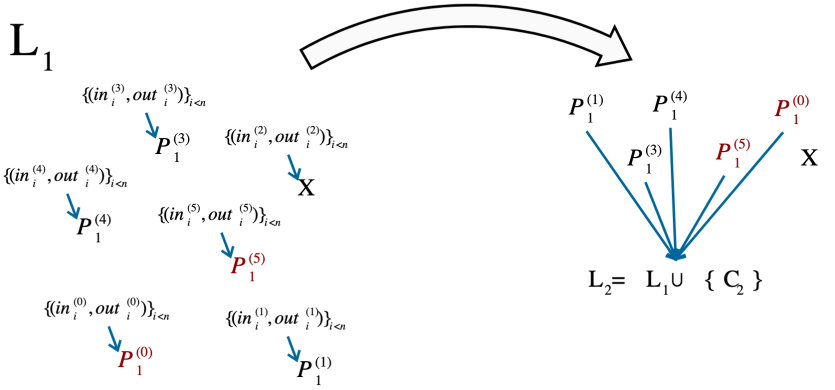
Exploration Compression Algorithm

Dechter, Malmaud, Adams & Tenenbaum (IJCAI 2013)



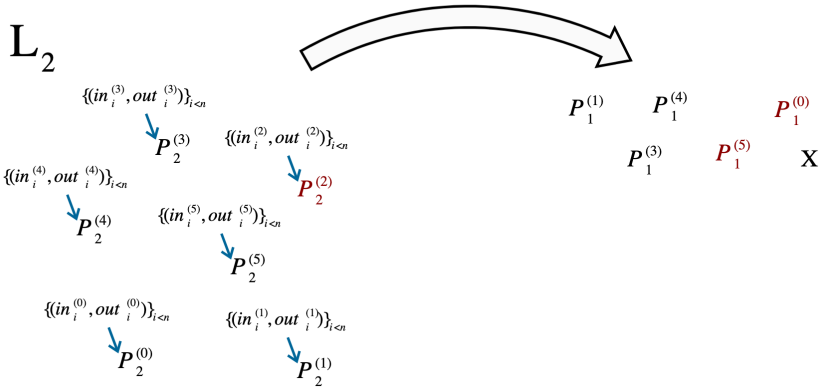
Exploration Compression Algorithm

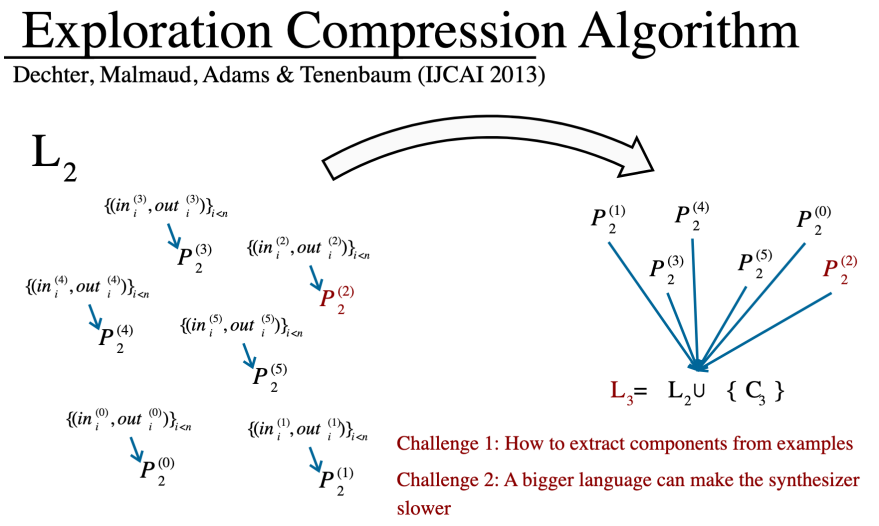
Dechter, Malmaud, Adams & Tenenbaum (IJCAI 2013)



Exploration Compression Algorithm

Dechter, Malmaud, Adams & Tenenbaum (IJCAI 2013)





Fast forward to 2014, with the growing interest in program synthesis, we started to see these ideas applied outside the domain of logic programs and in the context of more traditional programs. A student of Muggleton, Robert Henderson, wrote a thesis [3] that explored these ideas in the context of lambda calculus. At around the same time, here at MIT, Eyal Dechter, in collaboration with Jon Malmaud, Ryan Adams and Josh Tenenbaum showed some promising results applying these ideas in the context of functional programming [4]. The big limitation of these efforts, though, was in the efficiency of the component discovery algorithms.

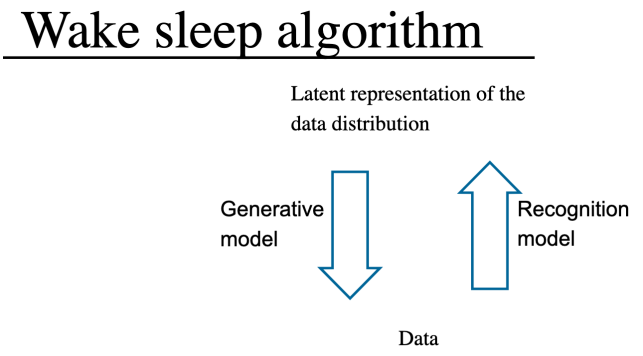
It was around this time that I started to collaborate with Kevin Ellis and Josh Tenenbaum on how we could apply the broader program synthesis toolkit in order to improve the effectiveness of the component discovery. This effort gave rise to the DreamCoder system, which we published in 2021 [5], and which in turn give rise to follow-up systems that aimed to improve the performance and the effectiveness of the component discovery process, such as Stitch [6] and Babble [7], which will be the main focus of this lecture.

9.2 DreamCoder

To understand the key ideas behind DreamCoder, it is useful to first understand the basics of Dechter et al. The approach is called the Exploration Compression algorithm (EC algorithm), and it works as follows. Given a corpus of programs, the algorithm first attempts to find solutions for each of the programs in the corpus during the exploration phase. Then, during the compression phase, the algorithm looks for common subexpressions in the synthesized programs and adds these as new components to the DSL. Each occurrence of the common

subexpression is then replaced with a call to the new component, making the programs more compact, and ideally making it easier to find solutions to future problems.

There were two main challenges with this approach. The first was that as the language grows, the branching factor of the search grows as well, so unless the new component is extremely useful, the synthesizer will actually get slower. The second problem was how to reliably discover useful components, especially when the necessary component is not a simple subtree that is common to many ASTs.



Wake stage: Real data goes through a fixed recognition model to produce the latent representation

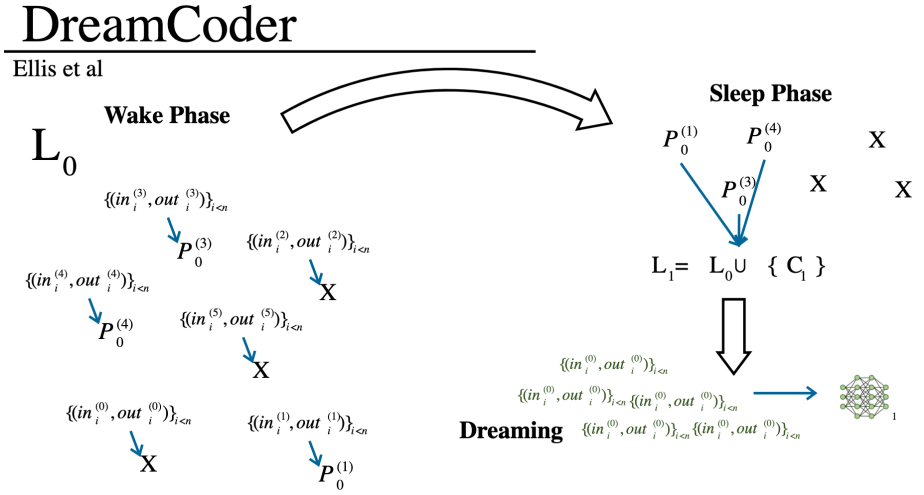
Sleep stage: Latent representations go through the fixed generative model to produce random data (dreams) to train the recognition model

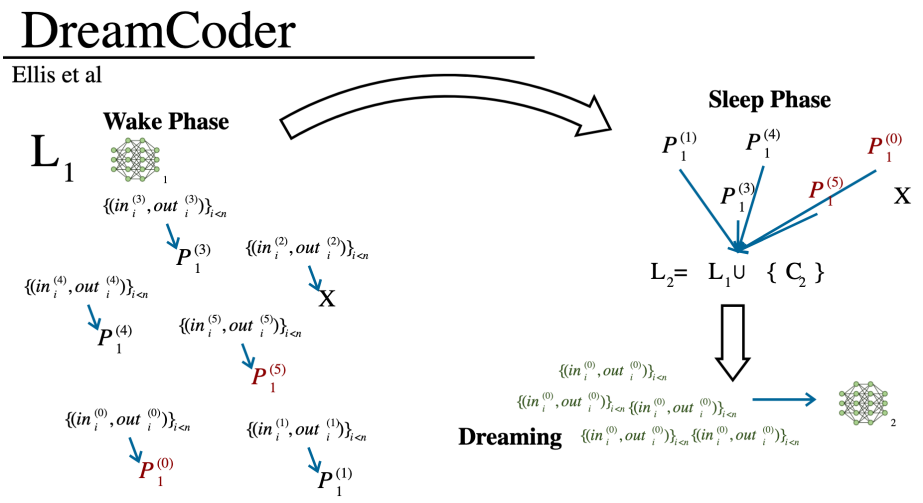
The DreamCoder system [5] aimed to address these two problems. The first problem with the branching factor was addressed by using a neural network to guide the search. This is where the "dream" in DreamCoder comes from. The approach is loosely inspired by the wake-sleep algorithm of Hinton et al for training generative models [8]. Although applied in a very different context, the idea behind the original wake-sleep algorithm is that you train a generative model and a recognition model in phases. During a wake phase, you take real observed data and use a recognition model to transform the data into a latent representation that captures the distribution of the data, and you use this to train a generative model to map from the latent representation back to the data. This is akin to learning from your observations of the real world when you are awake. During a sleep phase, you sample from the generative model to produce synthetic data — the dreams — and you use this to train the recognition model to map from the data back to the latent representation.

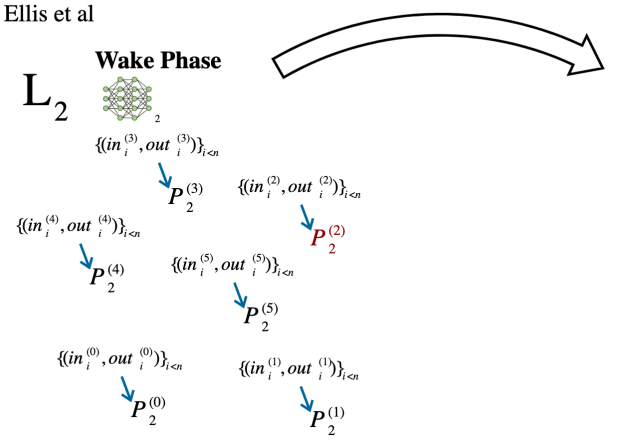
In DreamCoder, the observations are the input-output examples, the latent representation is a probabilistic grammar that describes a distribution over pro-

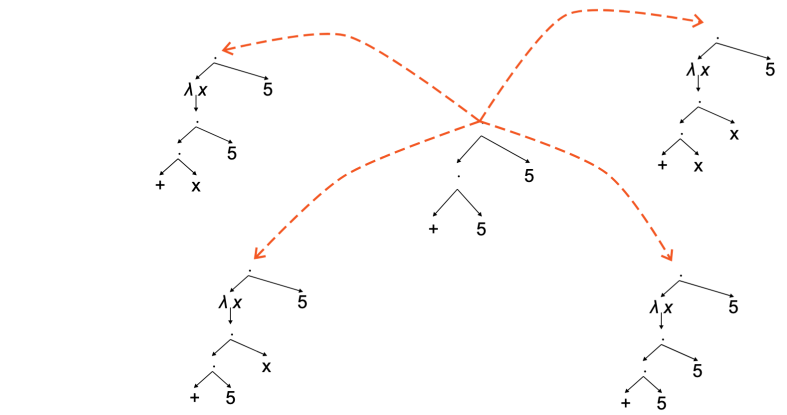
grams in the DSL. The recognition model is a neural network that takes as input the input-output examples and produces a distribution over programs in the DSL that are likely to satisfy the examples in the form of a probabilistic grammar like the ones we saw in the bottom-up search lecture. So the first big contribution of DreamCoder is to use the DSL to generate random programs and then generate input-output examples from these programs, the dreams that are then used to train the recognition model. The hope is that by training the recognition model on these synthetic examples, it will learn to generalize to real input-output examples and thus be able to guide the search more effectively.

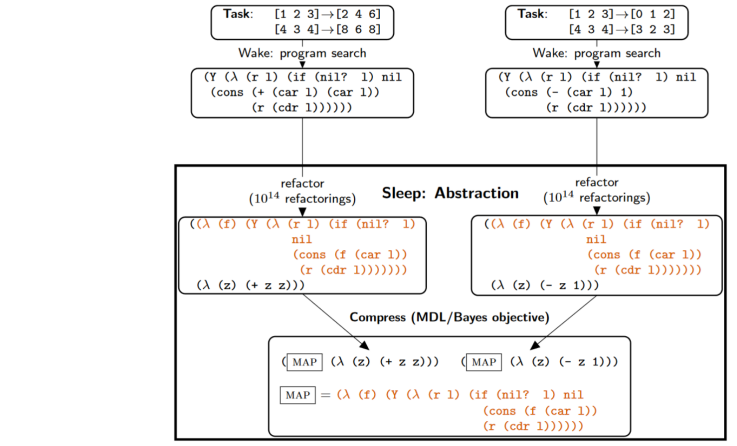
The DreamCoder algorithm is only inspired by the wake-sleep algorithm, though; it does not follow it exactly. The detailed algorithm is illustrated in the figure below. The algorithm actually has three phases. The waking phase corresponds to the exploration phase of the EC algorithm. This is where the system uses the current DSL and the current recognition model to try to solve each of the synthesis problems in the corpus. The sleep phase actually has two parts. The first part is the abstraction sleep phase; equivalent to the compression phase in the EC algorithm; this is where the system looks at the programs that were synthesized during the waking phase and tries to discover new components that can be added to the DSL. The second part is the dreaming phase, where the system generates synthetic input-output examples by sampling programs from the current DSL and executing them. These synthetic examples are then used to train the recognition model.




DreamCoder
Ellis et al
Wake Phase
L1
Sleep Phase
X
L2= L1∪ { C2 }
Dreaming


DreamCoder
Ellis et al
Wake Phase
L2





9.3 Component discovery in DreamCoder

The recognition model addresses the problem of the branching factor, but we still need to address the problem of reliably discovering useful components. For this, the DreamCoder paper used an approach based on e-graphs as explained in the lecture on version spaces. The key idea is that the original programs that were synthesized during the waking phase might not share many common sub-expressions, but they could be rewritten to expose more commonalities. By taking advantage of the e-graph's ability to compactly represent many equivalent expressions, we can discover more commonalities between

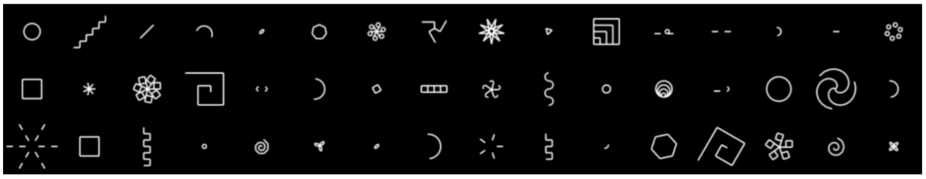
the programs and thus discover more useful components. E-grapsh are essential for this, because they can represent an exponential number of equivalent expressions compactly, and they also support efficient matching of patterns, so we can efficiently find common sub-expressions in the e-graph.

It is important to note however, that e-graphs only represent *semantically equivalent* programs. This means that if for a given synthesis problem $\{in_i^{(k)}, out_i^{(k)}\}$ there are two programs P_1 and P_2 that both satisfy the examples, but that are not semantically equivalent, there is a possibility that one of them (say P_1) can be rewritten to expose common sub-expressions with other programs in the corpus, while the other one (P_2) cannot. One way to deal with this is to actually synthesize multiple programs for each synthesis problem, and then create e-graphs for each of these programs. This is what DreamCoder does, and it is one of the reasons why the recognition model is so important: it allows us to efficiently explore the search space and find multiple programs for each synthesis problem. It also illustrates how the corpus of programs — and the components derived from it — serves a dual purpose: they help us solve the synthesis problems more efficiently, but they also help us generalize better to unseen inputs.

The DreamCoder paper showed the benefits of component discovery in a number of domains, including list processing, text editing, and turtle graphics. In each of these domains, the system started with a low-level DSL and was able to discover useful components that improved both the efficiency of the synthesis process and the generalization of the synthesized programs. The figure shows some example results from the paper. In the context of the turtle graphics domain, we can see how the system is able to discover high-order functions to draw radially symmetric patterns from lower-level drawing functions, and how it is able to build hierarchies of functions to draw complex shapes such as flowers and stars. The figure also shows how the dreams evolve over time as the DSL improves, from simple random squiggles to more complex and structured patterns.

Example: LOGO Graphics

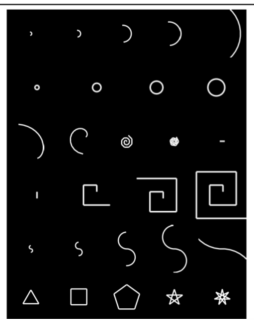
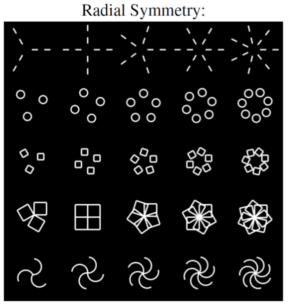
Input: Corpus of target shapes that we would like to learn how to draw



Input: Basic drawing language

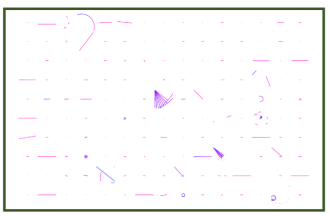
move, for, *, +, π, pen-up,…

Learned subroutines

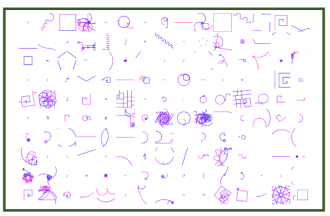
Parametric drawing routines in library		Higher-order drawing routine in library
Semicircle:		Radial Symmetry: 
Circles:		
Spiral:		
Greek Spiral:		
S-Curves:		
Polygons & Stars:		

Abstraction Dream

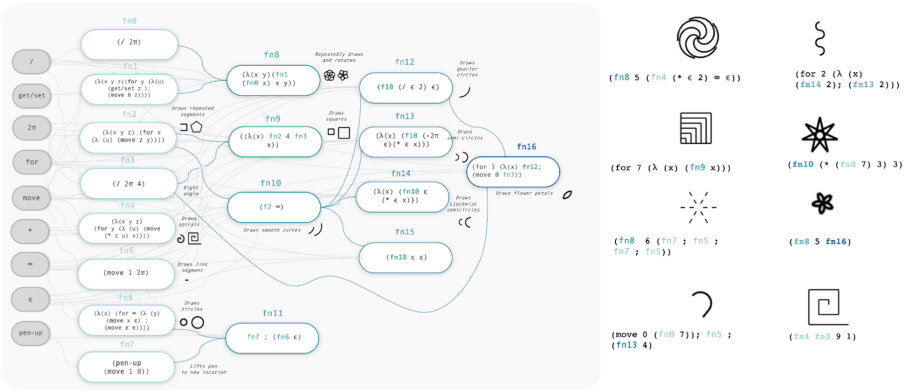
The model is trained by sampling from the learned language



Initially dreams are very unstructured



A richer language leads to more structured dreams

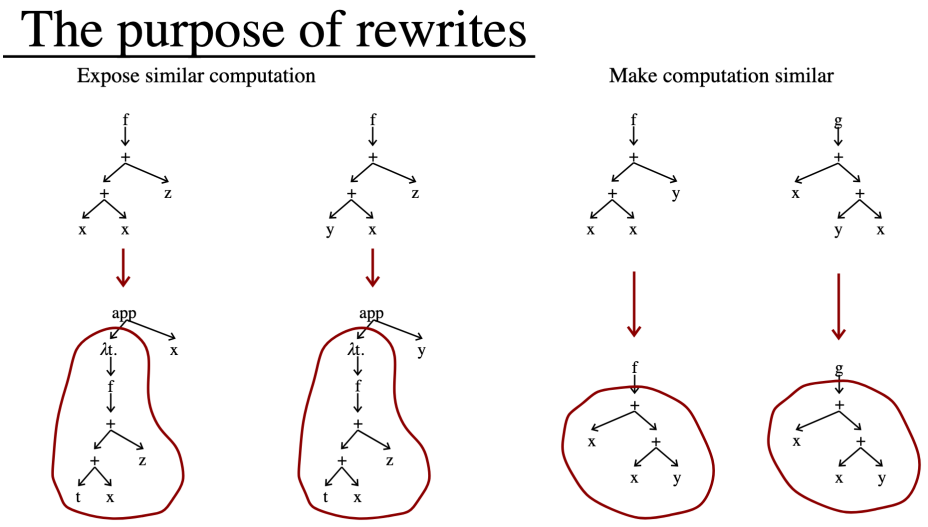


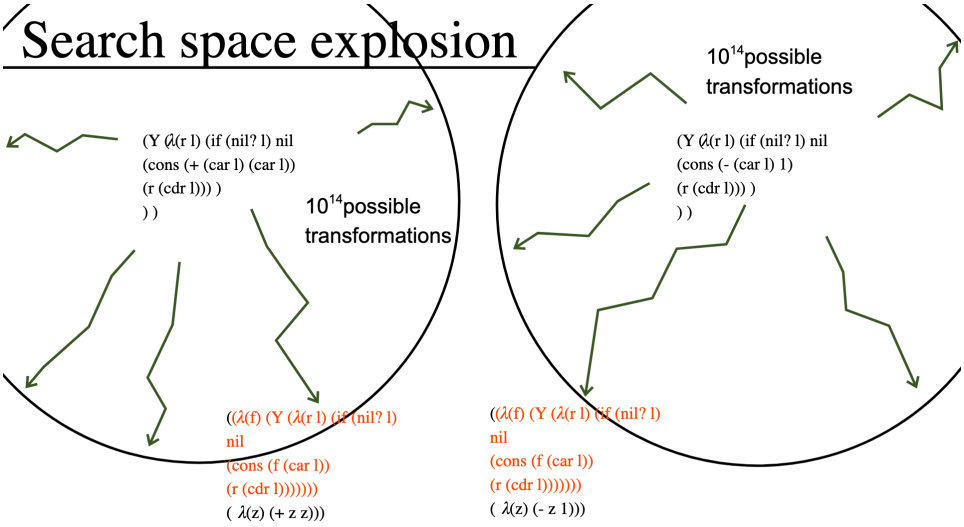
9.4 Stitch

There are a few major limitations of DreamCoder's component discovery algorithm. The most important one is that the e-graph based approach is computationally expensive, both in terms of time and memory. This limits the size of the programs that can be synthesized during the waking phase, which in turn limits the quality of the components that can be discovered.

An important observation about the DreamCoder component discovery algorithm is that the rewrites that give rise to the e-graph can actually serve two purposes. One goal is to simply expose common computations that are already syntactically identical but that are not recognized as common sub-expressions

because they are nested inside different contexts. This goal is extremely important, and it is satisfied by rules that introduce lambdas so that expressions in the middle of an AST turn into self-contained functions at the leaf of the tree that can become common sub-expressions. The other goal is to expose common structure that is not syntactically identical but that is semantically equivalent. This can be achieved through algebraic rewrite rules that exploit things like associativity and commutativity to rewrite expressions into different forms that might expose common sub-expressions. Within DreamCoder, however, the first set of rules already led to astronomically large spaces of equivalent expressions, so the second set of rules were largely avoided.

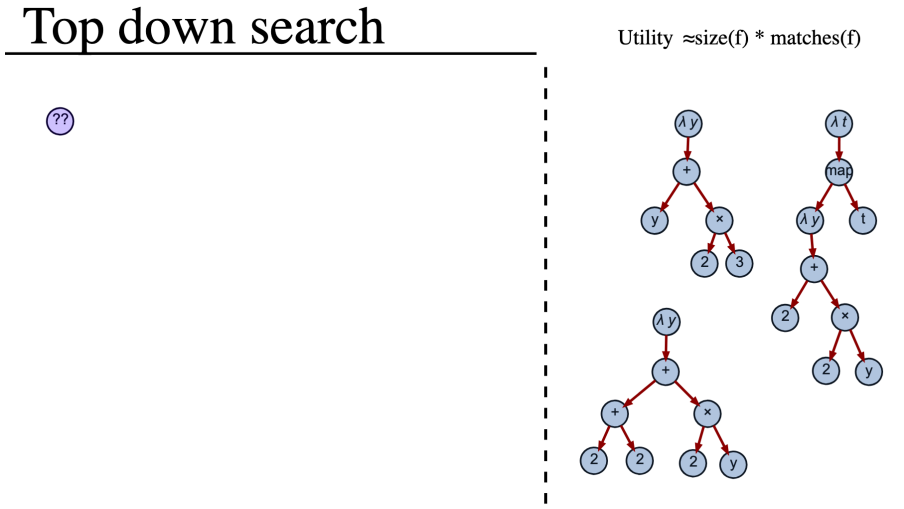




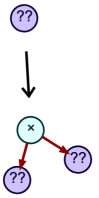
Stitch [6] is a follow-up system that focuses purely on the first goal, aiming to expose common sub-expressions that are already syntactically identical. By focusing purely on this goal, Stitch is able to use a much more efficient algorithm for component discovery that scales to much larger problems. The key insight behind Stitch is that instead of trying to rewrite the programs in the corpus to expose common sub-expressions, we can instead search for components directly using a top-down search algorithm guided by a utility function that measures how useful a component is in terms of how much it reduces the size of the programs in the corpus.

The core idea behind Stitch was simple. Per DreamCoder's intuition, the optimal components to add to the DSL are those that reduce the search space on future problems the most. While we cannot measure this, we can approximate it by instead looking at the programs we have already synthesized and measuring how much smaller they would have been if we had access to a new component. Thus, for a given component C, we can measure its *utility* $U(C)$ as the total size reduction of all programs in our corpus that would be achieved by substituting in C: $U(C) = \sum_{p \in P}(|p| - |p[C]|)$, where P is the set of programs in our corpus, $|p|$ is the size of program p, and $|p[C]|$ is the size of program p after rewriting it with component C. This utility function can be roughly approximated by counting the number of times C can be matched in each program p in the corpus, and multiplying that by the size of C. This is only a rough approximation as it fails to account for the fact that invoking C has a cost of its own that depends on things like the number of arguments it takes, or the fact that a component can reuse an argument in multiple places, which would reduce the size of the program more than simply replacing a subtree with a call to C. But as a first approximation, it works well enough.

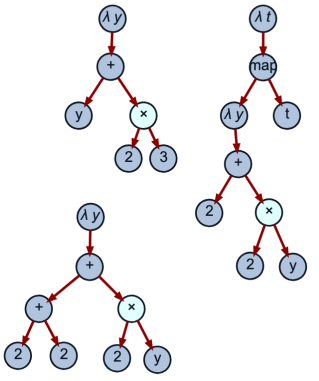
Stitch then uses this utility to guide a top-down search for new components. It starts with a component consisting of a single hole and then expands it by iteratively expanding holes in the component with non-terminal or terminal symbols from the DSL. Crucially, Stitch does not need to explore all branches of the top down search tree. The algorithm avoids this by using a branch-and-bound strategy that prunes branches that cannot possibly lead to a component with higher utility than the best component found so far. This is possible because at any point in the search, it is possible to compute upper and lower bounds on the utility of any component that can be constructed by expanding the current partial component. The lower bound is relatively simple: it is simply the utility of the current partial component, which can be computed by matching it against the programs in the corpus. The upper bound is conceptually also quite simple: it is the maximum possible utility that any component that can be constructed by expanding the current partial component can have. This process of computing upper and lower bounds and pruning branches whose upper bounds are lower than the best lower bound found so far is illustrated in the figure below.



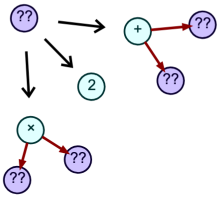
Top down search



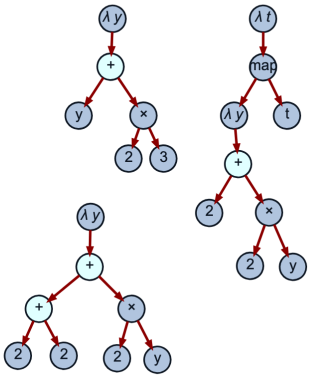
Utility ≈size(f) * matches(f)



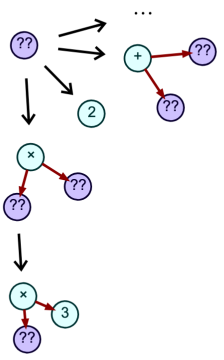
Top down search



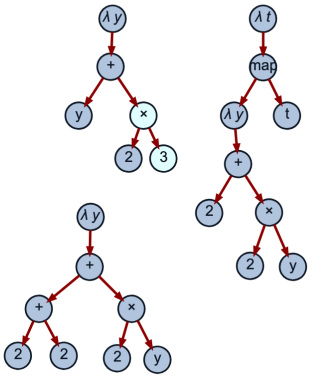
Utility ≈size(f) * matches(f)



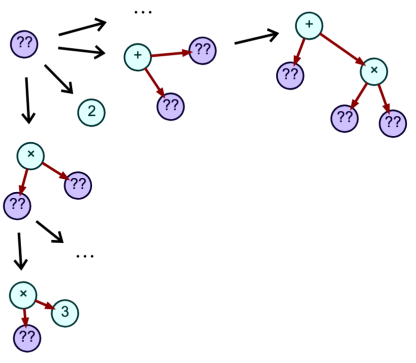
Top down search



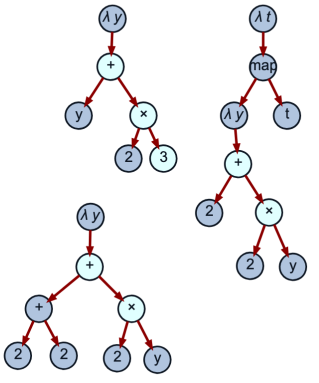
Utility ≈size(f) * matches(f)

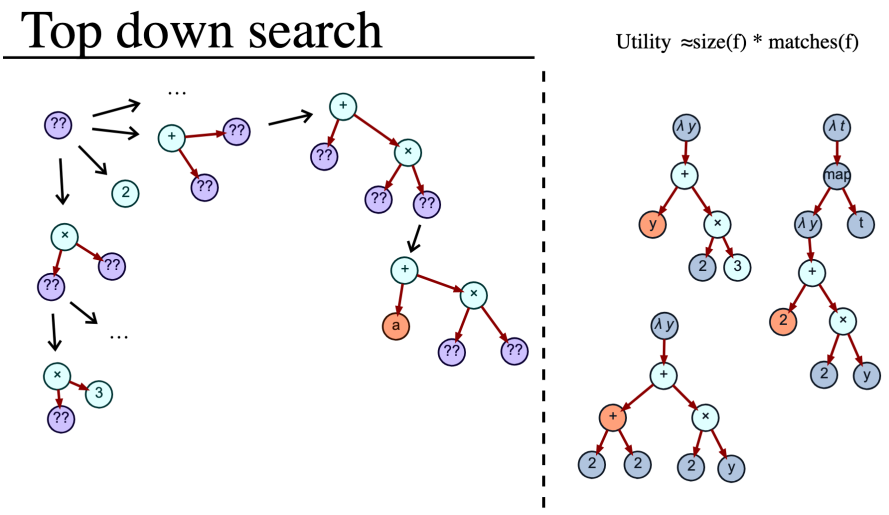


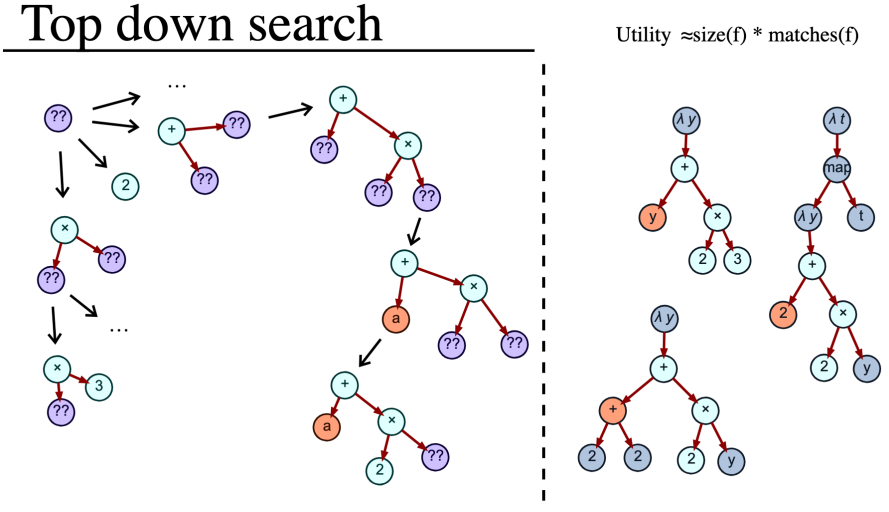
Top down search



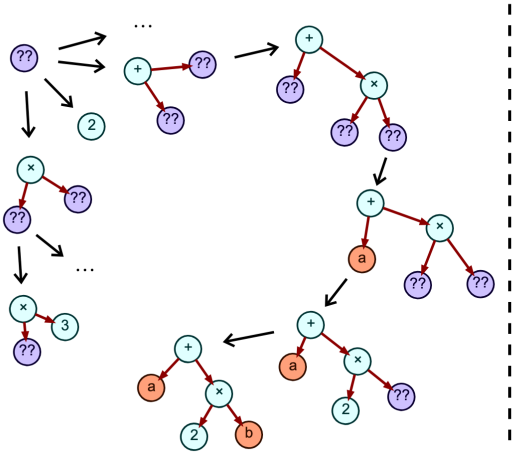
Utility ≈size(f) * matches(f)



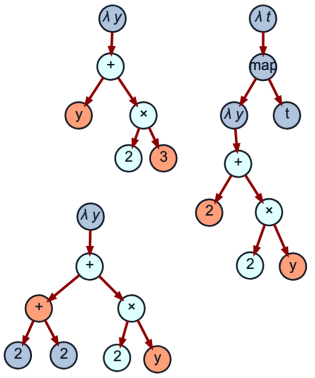

Top down search
Utility ≈size(f) * matches(f)


Top down search
Utility ≈size(f) * matches(f)

Top down search



Utility ≈size(f) * matches(f)



Branch and bound

For every node in the search tree, we can compute an upper and a lower bound for the utility of any leaf derived from that node

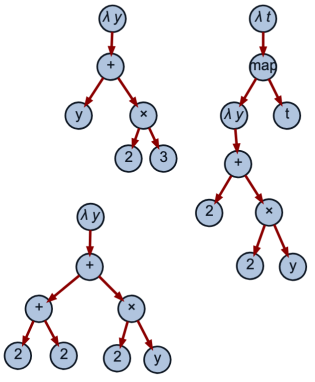
Any node whose upper bound is lower than the lower bound of another node can be pruned away.

Pruning

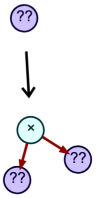


Bounds = [upper, lower]

Utility ≈size(f) * matches(f)

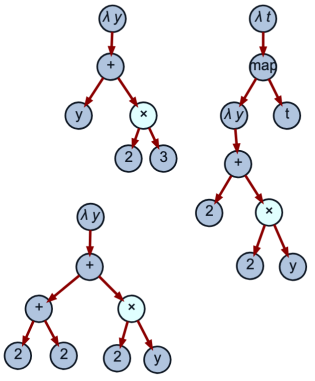


Pruning

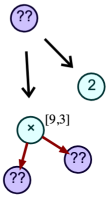


Bounds = [9, 3]

Utility ≈size(f) * matches(f)

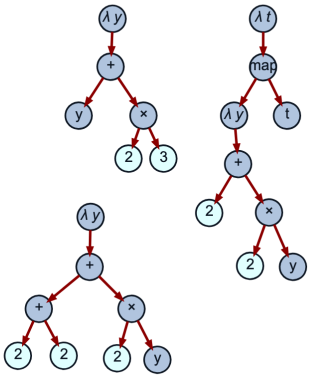


Pruning

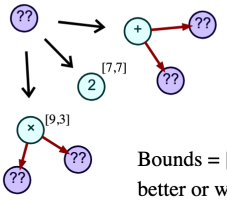


Bounds = [7, 7] so it could be better or worse

Utility ≈size(f) * matches(f)

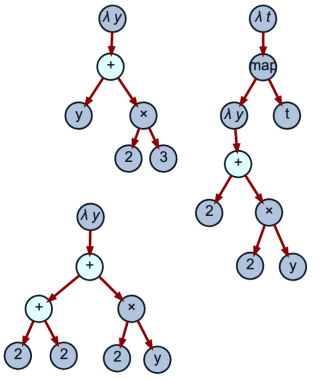


Pruning

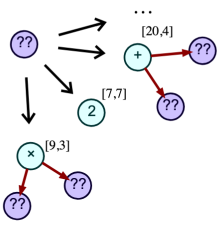


Bounds = [20, 4] so it could be better or worse

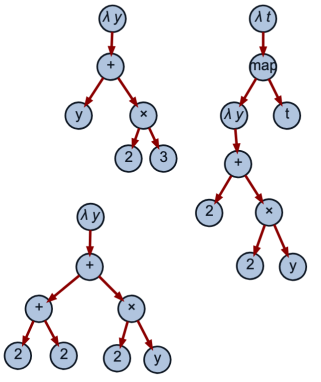
Utility ≈size(f) * matches(f)



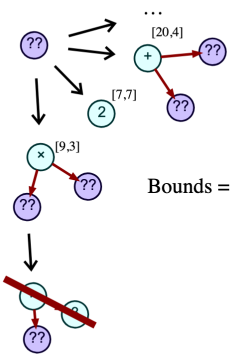
Pruning



Utility ≈size(f) * matches(f)

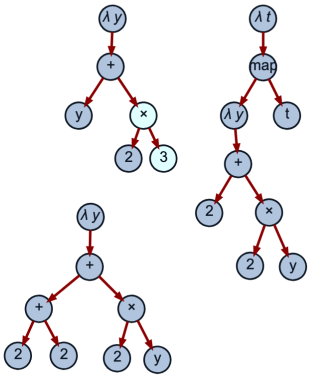


Pruning

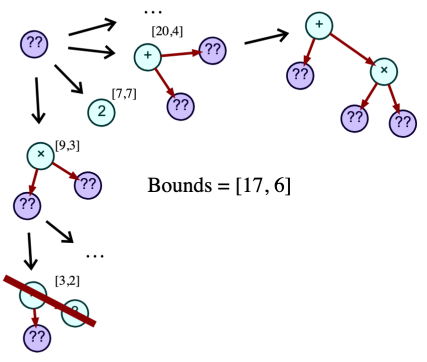


Bounds = [3, 2]

Utility ≈size(f) * matches(f)

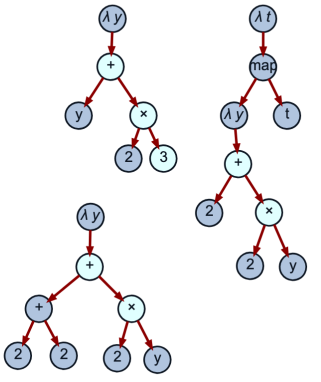


Pruning

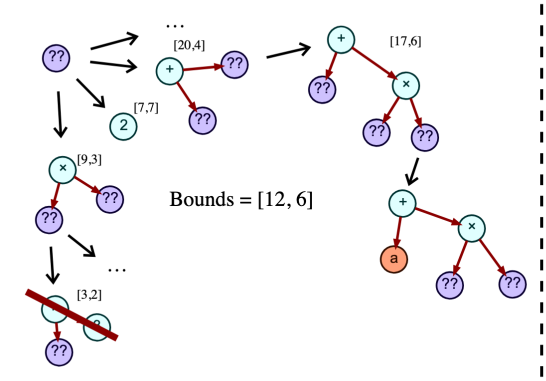


Bounds = [17, 6]

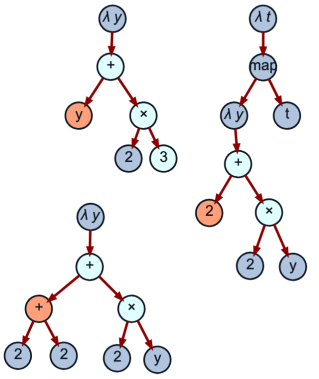
Utility ≈size(f) * matches(f)



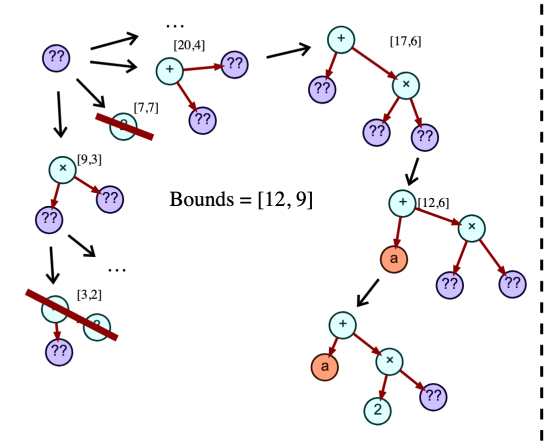
Pruning



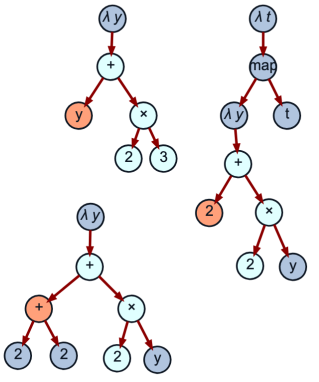
Utility ≈size(f) * matches(f)

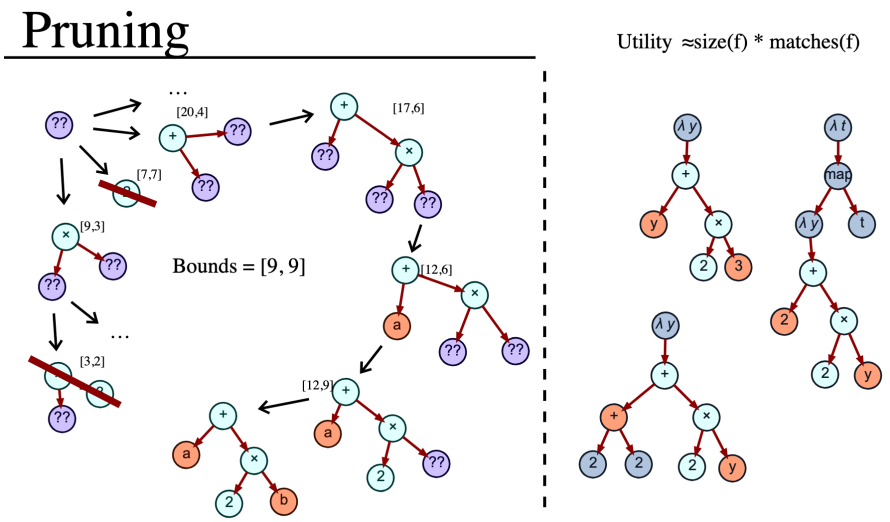


Pruning



Utility ≈size(f) * matches(f)





Stitch also had several advantages compared to DreamCoder, for example that it was an anytime algorithm: it could be interrupted at any time and still return a useful result, since it would always return the best component found so far. This, in addition to runtime and memory improvements on the order of 100x-10,000x, made Stitch a much more practical system for component discovery. However, Stitch did make some compromises in terms of the expressiveness of the components it could discover. Since the components were only matched against the programs on a *syntactic* level, it could not discover components that required more complex reasoning about the equivalance of expressions (the second kind of rewrite illustrated above). Bowers et al. were able to provide a proof-of-concept that remedied this by running Stitch *on top of a version space* constructed by rewriting, allowing to discover more components. But for domains that require aggressive rewriting to expose common sub-expressions, Stitch would not be able to discover them.

9.5 Babble

Contemporary with Stitch, another system called Babble [7] was developed, which took aim at supporting the full generality of DreamCoder's component discovery, but with better scalability, allowing it to support a fuller range of rewrite rules.

Babble's key technical insight was the development of Library Learning Modulo (Equational) Theories (LLMT), a component discovery algorithm that put semantic equivalence at the forefront. Alongside the DSL, the user provides a set of equational theories, which are sets of equations that describe equivalences between expressions in the DSL. For example, in a graphical DSL,

we might have an equation that states that no matter how we rotate a circle, it is still the same circle; or, in an arithmetic DSL, we might have an equation that states that $x + y - y = x$ for any x and y. Babble starts by repeatedly applying these rewrite rules to build up an e-graph that compactly represents many possible rewritings of the set of programs.

Once the e-graph has been constructed, the challenge is to extract useful components from it. To do so, Babble first generates a set of candidate components by applying anti-unfication to pairs of equivalence classes in the e-graph. At a high level, their anti-unification algorithm works by taking two equivalence classes and checking if they share a constructor; if they do, anti-unification is recursively applied to the sub-expressions of the constructor; if they don't, the expressions are replaced with a variable. Their key insight was that this procedure can be applied to all pairs of equivalence classes efficiently through a bottom-up dynamic programming approach. This process generates a set of candidate components which are then added to the e-graph as additional rewrite rules. Finally, Babble frames selecting the optimal set of components as extracting the smallest possible corpus of programs from the e-graph. LLMT allows babble to discover sets of multiple components at once that work well together, and also allows it to reason past syntactic differences to find useful components that would not be discovered by other approaches.

Overall, component discovery is a useful tool in the overall toolkit of program synthesis which has proven to be effective not only in the context of search-based synthesis techniques, but also in the context of Language Models that we will study in the next unit.

References

[1] Stephen Muggleton. Inductive logic programming. *New Generation Computing*, 8(4):295–318, 1991.

[2] Stephen Muggleton. Duce, an oracle based approach to constructive induction. In *Proceedings of the 10th International Joint Conference on Artificial Intelligence (IJCAI-87)*, pages 287–292, Milan, Italy, 1987. Morgan Kaufmann.

[3] Robert John Henderson. *Cumulative Learning in the Lambda Calculus*. PhD thesis, Imperial College London, London, UK, 2014. URL `https://spiral.imperial.ac.uk/server/api/core/bitstreams/4a8f6ab5-3bf0-41e4-8b58-8001265c4290/content`.

[4] Eyal Dechter, Jonathan Malmaud, Ryan P. Adams, and Joshua B. Tenenbaum. Bootstrap learning via modular concept discovery. In Francesca Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pages 1302–1309. IJCAI/AAAI, 2013. URL `http://www.aaai.org/ocs/index.php/IJCAI/IJCAI13/paper/view/6890`.

[5] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke B. Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dreamcoder: bootstrapping inductive program synthesis with wake-sleep library learning. *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021.

[6] Matthew Bowers, Theo X Olausson, Lionel Wong, Gabriel Grand, Joshua B Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages*, 2023.

[7] David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. babble: Learning better abstractions with e-graphs and anti-unification. *Proceedings of the ACM on Programming Languages*, 2023.

[8] Geoffrey E. Hinton, Peter Dayan, Brendan J. Frey, and Radford M. Neal. The "wake-sleep" algorithm for unsupervised neural networks. *Science*, 268 (5214):1158–1161, 1995. doi: 10.1126/science.7761831.